**MULTIMODAL INTERACTIVE TECHNOLOGIES AND APPLICATIONS**

# Prototype Architecture

Botanical Garden


| Group Name | Gardeners |
|---|---|
| Kaifei Xu | 11115439 |
| Keyan Cheng | 11154362 |
| Shiqi Huang | 11111452 |
| Yufei Liu | 11118413 |


May 25, 2026

# 1 Architectural Overview

Botanical Garden is implemented as a browser-based prototype with a lightweight local backend. The frontend provides the immersive garden experience, plant selection, speech interaction, visual feedback, and information panels. The backend provides optional LLM-assisted intent parsing and response generation, while the frontend keeps a deterministic local fallback so that the main interaction loop can still work when the LLM service is unavailable.

At a high level, the prototype is divided into four layers:

1. **Immersive scene layer:** renders the panoramic garden, 3D plant models, and spatial highlights.
2. **Interaction layer:** captures pointer-based plant selection, typed input, speech-to-text input, visible-plant ambiguity detection, and ambiguity-resolution actions.
3. **Dialogue and grounding layer:** maps user utterances to intents, connects those intents to the currently selected plant or recently visited plants, and decides which UI response should be shown.
4. **Knowledge and generation layer:** stores structured plant records and, when enabled, uses the backend LLM service to generate more natural spoken explanations.

The architecture intentionally keeps the most important interaction state on the client side. This makes the prototype responsive and allows plant selection, local intent parsing, panel rendering, and fallback behavior to work without a persistent server session.

# 2 Frontend Architecture

The frontend is a Vite application written in modular JavaScript. The entry point is `src/main.js`, which initializes global UI controls, speech recognition, plant selection, scene navigation, and the first garden scene.

The main frontend modules are:

- `src/scene/*`: renders the 360-degree garden scenes, 3D plant models, and plant selection in the immersive scene.
- `src/app/*`: owns interaction state and high-level behavior such as identifying a plant, querying an attribute, comparing plants, and resolving ambiguity.
- `src/intent/*`: parses user utterances locally and optionally delegates to the backend intent API.
- `src/ui/*`: renders the plant information panel, medical panel, comparison panel, fallback panel, ambiguity overlay, and query controls.
- `src/speech/speech.js`: integrates browser speech recognition and text-to-speech output.
- `src/data/*`: contains local scene, plant, and intent data used by the prototype.

The frontend state is centralized in `src/app/state.js`. It tracks the active panoramic scene, the currently selected plant, the set of plants currently visible in the user's view, ambiguity candidates derived from those visible plants, the ordered list of visited plant ids, and the speech recognition failure count for retry behavior.

This state is small but important. It allows the system to interpret phrases such as "this plant", "the previous one", or "compare it with lavender" against the user's actual interaction history instead of treating each utterance as an isolated command.

# 3 Backend Architecture

The backend is a minimal Node.js HTTP server located in `server/index.js`. It exposes JSON endpoints that support the frontend dialogue flow:

- `GET /api/health`: health check and configured model information.
- `POST /api/parse-intent`: parses a user utterance into an intent and slots.
- `POST /api/generate-info`: generates a spoken answer for one plant.
- `POST /api/generate-disambiguation`: generates a clarification prompt for multiple candidate plants.
- `POST /api/clarify`: resolves a user's answer to an ambiguity prompt.
- `POST /api/compare`: generates a comparison response for two plants.

The backend is intentionally stateless from the frontend's perspective. Each request includes the context needed for that turn, such as the utterance, selected plant, visited plant ids, candidate plants, or dialogue history. This keeps the prototype simple to run locally and avoids hidden server-side state that would be hard to debug during evaluation.

The backend services are split by responsibility:

- `server/services/intentParser.js`: deterministic parser fallback.
- `server/services/llm.js`: LLM-backed parsing and response generation.
- `server/services/plantDb.js`: reads the plant database from `src/data/plants.json`.
- `server/handlers/*`: request handlers for query, clarify, and compare flows.

If the LLM call fails, the server returns a local fallback or an empty generated speech field. The frontend can still render structured cards from local plant data, so the visual part of the interaction remains available.

# 4 Core Interaction Flow

The most common interaction starts when the user looks at a plant area and asks a question.

1. The system checks the plants that are currently visible in the user's view.

2. If only one plant is visible or clearly selected, the frontend records it as the current plant; if several visible plants could match the user's reference, the frontend opens an ambiguity flow.

3. The user submits text through typing or browser speech recognition.

4. `handleQuery()` sends the utterance and current context through the intent parser.

5. The parsed intent is routed to one of the frontend handlers: identification, attribute query, comparison, ambiguity resolution, or unknown intent.

6. The selected plant is visually highlighted in the 3D scene.

7. The corresponding 2D panel is rendered: botanical information, medicinal information, comparison result, fallback options, or ambiguity choices.

8. Text-to-speech reads the concise answer or clarification prompt.

For ambiguous references, the frontend stores the visible candidate plants in state and shows an A/B/C overlay. The user may resolve the ambiguity by clicking an option or by speaking a phrase such as "A" or the plant name. As a fallback option, users can directly click a plant model to make an explicit selection.

For comparison, the frontend combines the current selected plant with the visited plant history. This enables contextual queries such as "Is this more drought tolerant than the previous one?" The comparison handler resolves the second plant, chooses the relevant attribute, and renders a side-by-side comparison panel.

# 5 Data Model and Assets

Plant knowledge is stored as structured JSON in `src/data/plants.json`. Each plant record contains identifiers, display names, aliases, descriptions, attributes, and optional information such as medicinal value, toxicity, or drought tolerance. Scene metadata is stored in `src/data/scenes.json`, while intent-related rules are stored in `src/data/intents.json`.

Visual assets are kept in the `public/` directory:

- panoramic scene images;
- 3D plant models in `.glb` format;
- UI assets.

This separation keeps the prototype extensible. New plants can be added by combining a plant data record, a model asset, and scene placement metadata, without changing the main interaction code.